CS6650 Final Report

Group Members: Nithish Bhat, Jingjing Pan, Hong Guo, Stella Zhang

Presentation Video: Distributed Sports Ticket Reservation System

Project Repo: Click Here

MeisterTask dashboard: Click Here

Objectives

Short-term

- Build a working ticket reservation system that handles concurrent purchases without overselling.
- Compare two concurrency control strategies (pessimistic vs. optimistic locking) under realistic load and measure the difference.
- Demonstrate horizontal scaling on ECS and find where throughput stops improving.
- Show the system recovers gracefully from a service instance crash mid-load.

Long-term

- Add a Redis-based distributed lock as a third concurrency strategy, comparing in-database locking against an external lock manager. This requires adding ElastiCache to our Terraform config and lock acquisition code in the order-service.
- Introduce a Redis caching layer in front of the ticket-service to reduce read pressure on Postgres under high browser traffic.

Related work

Our project builds on course material and weekly assignments around scalable web services and concurrency control. The microservices pattern we use – separate read and write services behind a load balancer – comes directly from the course lectures on distributed system design. Concepts like horizontal scaling with ECS, ALB, and database connection pooling all trace back to topics we covered in class. The locking comparison (pessimistic vs. optimistic) is grounded in database concurrency theory we studied, and the idea of idempotency keys came from a weekly mock session with our TA, where we realized that in a reservation system, network retries under heavy load are inevitable and duplicate submissions need to be handled at the application level.

Methodology

Architecture

Two Go microservices run on ECS:

ticket-service: read operations – browse all tickets, get one ticket, health check. Runs database migrations on startup.
order-service: write operations – place an order with an idempotency key. Locking strategy is selected via the `LOCK_STRATEGY` environment variable.

Both connect to a single RDS PostgreSQL 15 instance. The ALB routes `/tickets*` to ticket-service and `/orders` to order-service.

Concurrency Control

Pessimistic locking uses `SELECT ... FOR UPDATE` inside a database transaction. This locks the ticket row so other transactions block until the first one commits. It guarantees serialized access but creates a queue under contention.

Optimistic locking skips the explicit lock. It does a conditional `UPDATE ... WHERE tickets_reserved + qty <= quantity`. If 0 rows are affected, another transaction modified the row first. The service retries up to 3 times with exponential backoff (50ms, 100ms, 200ms) plus random jitter. Faster under low contention, but retry overhead adds up under high contention.

Both are backed by a `CHECK (tickets_reserved <= quantity)` constraint on the tickets table – Postgres rejects an oversell at the storage layer regardless of application logic.

Idempotency

Every order request includes an `idempotency_key`. Before doing anything, the order-service checks if that key exists in the orders table. If it does, it returns the previous result. This means network retries or duplicate submissions won't double-book.

Experiments

Experiment 1 – Horizontal Scaling: 200 concurrent users, order-service task count varied (1, 2, 4, 8). Traffic spread across all 4 ticket types (low contention). Measures where throughput stops improving as we add compute.

Experiment 2 – Locking Strategies

- Pessimistic lock: lock the row first, then do the update. Safe but everyone has to wait in line.
- Optimistic lock: just try the update, and if someone beats you to it, retry. Fast when there's no competition.
- Redis lock: use Redis as an external lock before touching the database.

Experiment 3 – Failure Recovery: 10-minute test, manually kill one ECS task mid-run with `aws ecs stop-task`. Observe recovery time, error impact, and data integrity.

Experiment 4 – Data Consistency Under Failure: from the experiment 3 results, we confirmed no overselling and no data corruption – tickets_reserved never exceeded quantity for any ticket, and the RDS CHECK constraint held throughout.

Preliminary Results

Experiment 1 – Horizontal Scaling

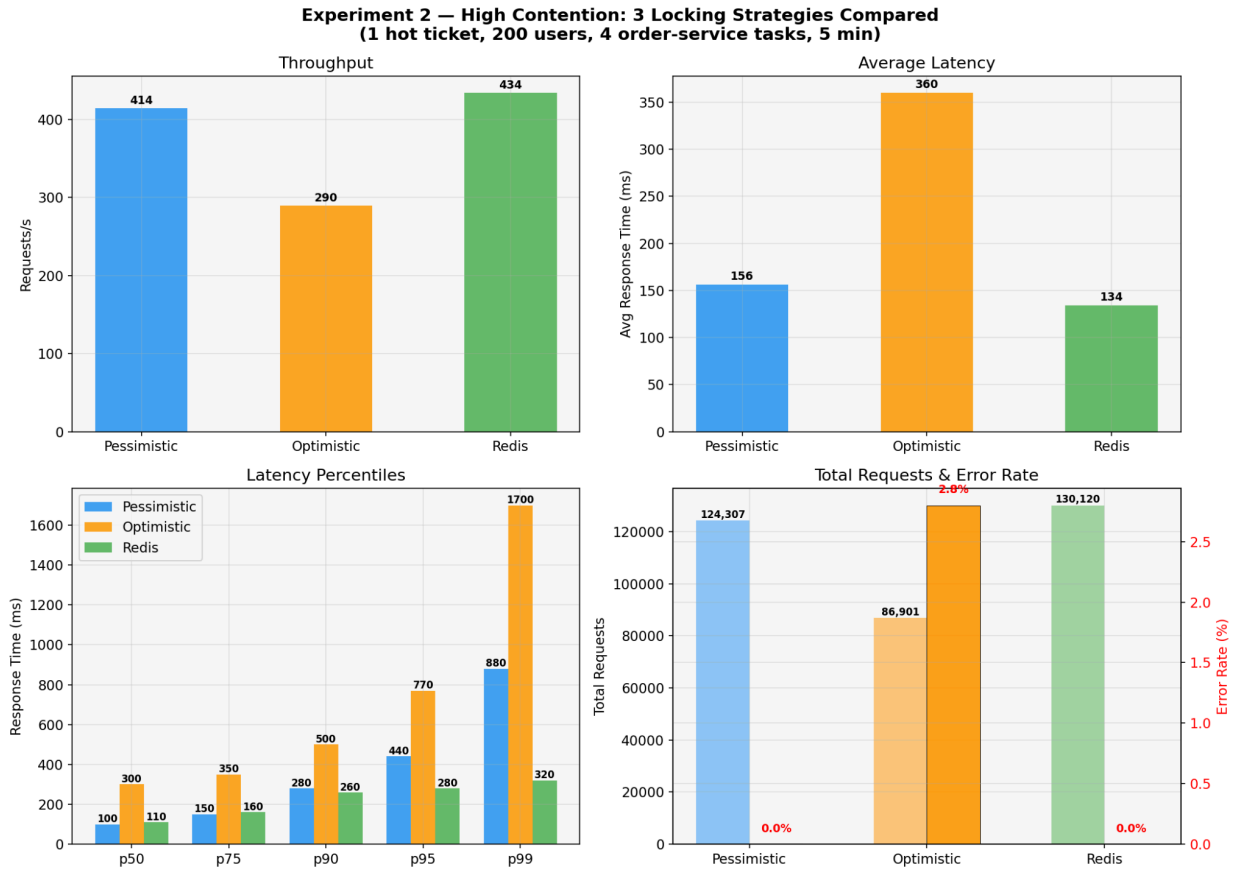
200 users, spawn-rate 20, 5 minutes per run, pessimistic locking, traffic spread across 4 tickets.

Tasks	Total Reqs	Req/s	Avg (ms)	p50 (ms)	p99 (ms)	Errors
1	109,538	365	238	200	930	0%
2	108,986	363	241	94	860	0%
4	153,617	512	84	76	320	0%
8	155,159	517	80	76	160	0%

The big jump is from 2 to 4 tasks – throughput goes from 363 to 512 req/s and p99 drops from 860ms to 320ms. Going from 4 to 8 barely moves throughput (512 to 517) but does cut the p99 tail in half (320 to 160ms). At that point the single Postgres instance is the bottleneck, not the application layer. Zero errors across all configurations.

Experiment 2 – Locking Strategies

High Contention (1 hot ticket, 200 users all fighting for it):



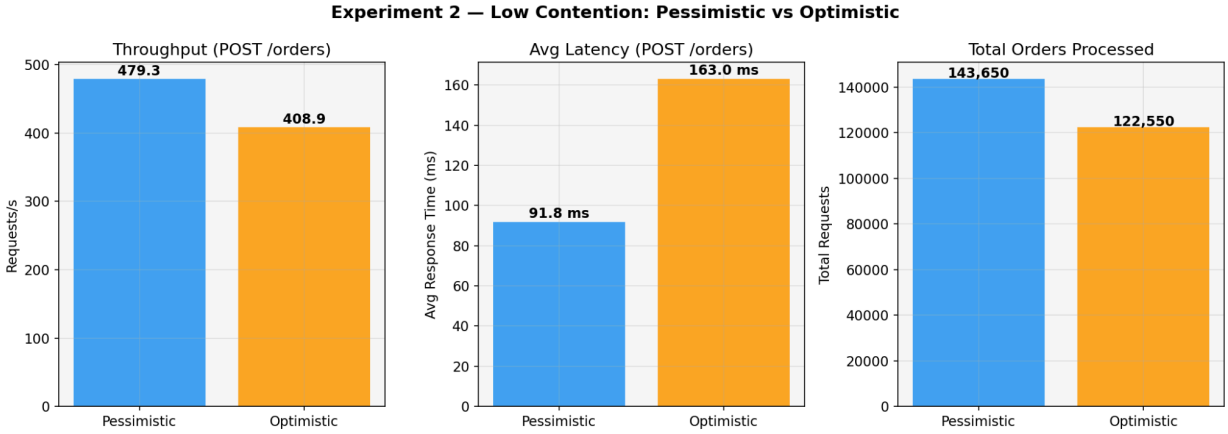
Lock Strategy	Total Reqs	Req/s	Avg (ms)	p50 (ms)	p99 (ms)	Errors
Pessimistic	130,526	435	152	100	860	0%
Optimistic	91,161	304	347	300	1,600	2.67%
Redis Lock	130,120	434	134	110	320	0%

Redis Lock had the best overall performance — lowest average latency (134ms), lowest p99 (320ms), and zero errors. It handled contention well by moving the lock outside the database, reducing row-level blocking in PostgreSQL.

Pessimistic Locking was close behind in throughput (435 req/s vs 434 req/s) and also had zero errors. The tradeoff is higher tail latency — p99 hit 860ms because requests queue up waiting for the row lock. It's the simplest and most reliable option if you don't want to add Redis as a dependency.

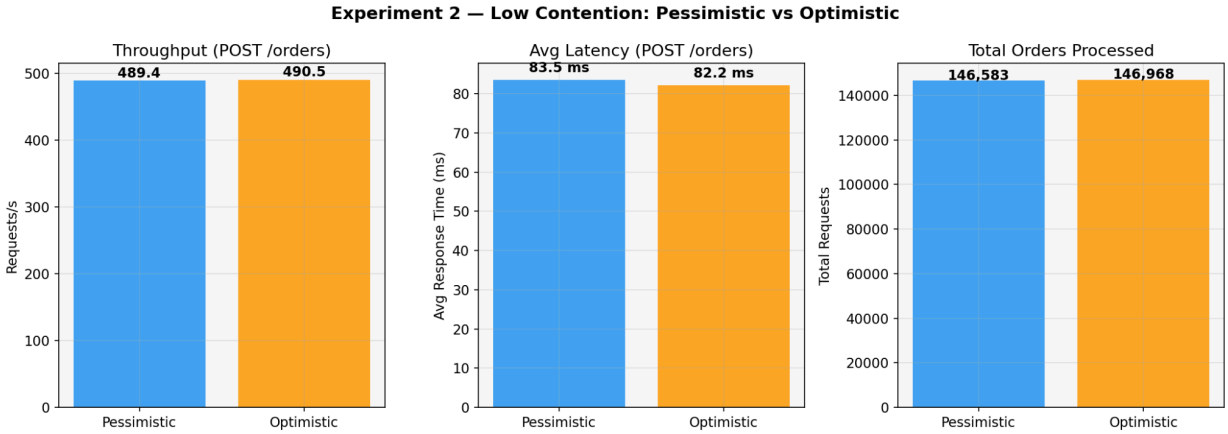
Optimistic Locking struggled the most. Throughput dropped to 304 req/s, and 2.67% of requests failed. When 200 users all fight for the same ticket, the retry logic can't keep up — too many conflicts cause retries to pile up, which increases latency (347ms avg) and eventually leads to failures after max retries are exhausted.

Low Contention - Attempt 1 (200 users, 4 Events)



Looking at the numbers, the pessimistic approach still beat the optimistic one—throughput was about 17% better and latency was 44% lower. I realized that having 50 people trying to write to the same event at once is still way too much for optimistic locking. The system kept hitting those retry delays (waiting up to 200ms), which really slowed things down at the end.

Low Contention - Attempt 2 (200 users, 50 Events)



Performance was virtually identical — less than 0.3% difference in throughput and latency.

Key Findings from above low contention experiments

1. Contention threshold matters more than strategy choice. At ~4 concurrent writers per row, the pessimistic lock's SELECT FOR UPDATE blocking and the optimistic lock's retry mechanism both become nearly irrelevant — most operations succeed on the first attempt without waiting.

2. Optimistic locking's weakness is specifically retry amplification. Under high contention (200:1), retries cascade — each failed attempt waits 50–200ms, inflating p95 to 770ms and p99 to 1,700ms. Under low contention (4:1), retries rarely trigger, and optimistic actually edges ahead slightly (p99: 240ms vs 270ms) because it avoids acquiring row-level locks entirely.

Experiment 3 – Failure Recovery

200 users, 10 minutes, 4 tasks, pessimistic locking. Manually killed one ECS task at roughly the 5-minute mark.

Metric	Total Reqs	Req/s	Avg Latency	p99 Latency	Errors
Value	270,876	451	139 ms	450 ms	0%

Zero errors across the full 10 minutes, including through the crash. The Locust chart shows a brief latency spike around the kill point, but it settles within about 30 seconds. ECS detected the failed task, the ALB deregistered it, and a replacement task launched automatically. The remaining 3 tasks absorbed the load during recovery.

Experiment 4 - Cache

Cache was added to the ticket-service as a read-aside layer backed by the existing ElastiCache Redis cluster. GET /tickets and GET /tickets/{id} check Redis first and fall back to PostgreSQL on a miss, writing the result back to Redis with a 10-second TTL. The ENABLE_CACHE environment variable toggles the layer on and off via Terraform, so both runs used identical infrastructure — only the caching behavior differed. POST /orders and GET /validate always bypass the cache and hit PostgreSQL directly to ensure order consistency and accurate overselling checks.

Users: 200, ramp up: 20, runtime: 2m

Metric	Total Reqs	Req/s	Avg (ms)	Max(ms)	p50 (ms)	p99 (ms)	Errors
Without Cache	88,420	736.7	86.2	2033	80	180	0%
With Cache	90,058	750.3	80.8	592	80	95	0%
% Change	+1.9%	+1.8%	-6.3%	**-70.9%**	0%	**-47.2%**	—

Key takeaway:

- Tail latency (p99) cut by 47% (180ms → 95ms) — cache absorbs repeated read traffic and eliminates DB contention spikes under load.
- Max latency dropped 71% (2,033ms → 592ms) — Redis eliminates the worst-case DB round trips that drive outlier response times.

- DB load reduction visible on non-cached endpoints — POST /orders max fell from 1,980ms to 592ms and GET /validate max fell from 2,033ms to 326ms despite neither using the cache, showing the DB was under less pressure overall.
- Median held flat at 80ms — Redis within the same VPC has similar baseline latency to PostgreSQL for small payloads, so the benefit surfaces in the tail, not the median.
- Throughput gain was modest (+1.8%, 736.7 → 750.3 req/s) — the write path (POST /orders) remains DB-bound, capping overall RPS gains even as read latency improved.

Impact

Ticket overselling is a real-world problem (anyone who's tried to buy concert tickets knows this). The tradeoffs between locking strategies are usually discussed in theory – our project puts concrete numbers on them. Pessimistic locking handled 43% more throughput than optimistic under high contention, with 0% errors vs. 2.67%. That's a useful data point for anyone building a reservation system and wondering which approach to pick.

The scaling experiment answers a practical question: how many service instances do you actually need? For our setup(200 users), the answer is about 4 – beyond that, the database is the bottleneck and adding more application instances doesn't help much. Knowing where the wall is matters for capacity planning and cost.

Other teams in the class could point their own Locust scripts at our ALB if they want to test cross-system traffic patterns, and our Locust script is parameterized (host URL, user count, hot-ticket toggle) enough to work against any similar order-placement API. So if another team has a similar API that accepts POST requests with a ticket ID and quantity, they could use our Locust script against their system too – just change the host URL.

Project Management

We broke the project down with the help of AI, which helped us split the work into four clear phases: Initiation, Planning, Resources & Execution, and Monitoring & Closing. Each phase had its own deliverables — defining system scope and guarantees, designing the architecture, setting up AWS infrastructure and load testing, then implementing and monitoring the core services. We divided tasks equally across team members based on strengths, tracked everything on this Kanban board, and used it throughout the project to monitor progress, catch blockers early, and re-balance work when something ran over. The board made ownership visible: every card has an assignee and a due date, so we always knew who owned what and what was at risk.